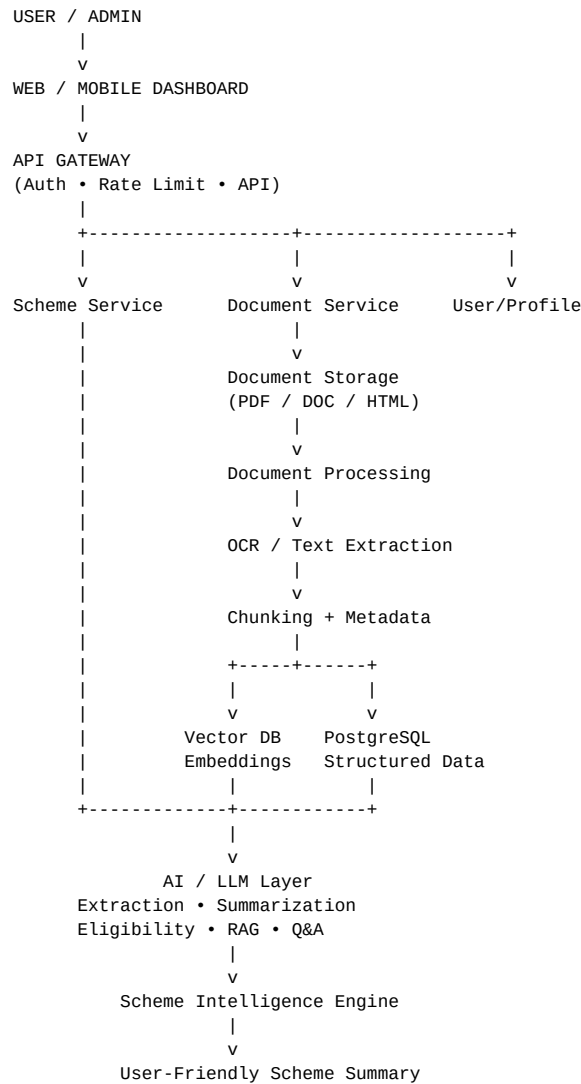


Government Scheme Intelligence Platform

Project Architecture & Technical Design

An AI-powered platform that reads government scheme documents, extracts important information, explains it in simple language, and helps users determine which schemes may be relevant to them.

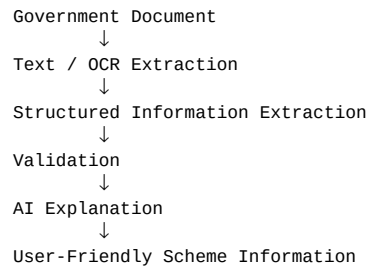
1. High-Level Architecture



2. Core Concept

Do not treat the project as only a PDF summarizer.

Recommended pipeline:



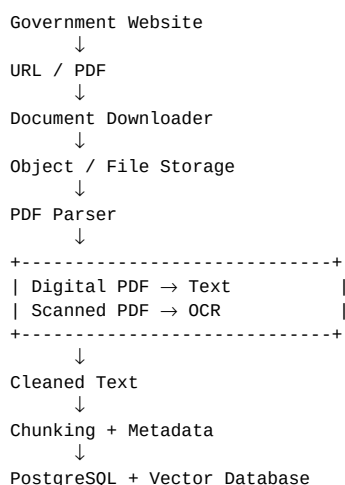
This makes the system more reliable and allows the frontend to display consistent fields.

3. Information to Extract

Each scheme should be converted into structured data such as:

- Scheme name
- Ministry / department
- Objective
- Target beneficiaries
- Benefits
- Eligibility criteria
- Age requirements
- Income requirements
- State / location requirements
- Occupation requirements
- Financial assistance
- Documents required
- Application mode
- Application steps
- Deadline
- Important conditions
- Official source
- Document date / version
- Page-level citations

4. Document Ingestion Pipeline



5. AI Processing Layer

A. Scheme Classifier

Classifies schemes into categories such as Agriculture, Education, Employment, Healthcare, Women & Child, Housing, Finance, MSME, Social Security, etc.

B. Information Extractor

Extracts benefits, eligibility, documents, application process, deadlines and important conditions.

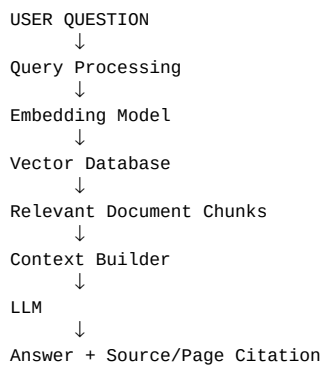
C. Eligibility Engine

Compares a user's profile with the explicit eligibility rules of a scheme.

D. RAG / Q&A;

Retrieves relevant document sections and asks the LLM to answer using those sections, preferably with source and page citations.

6. RAG Architecture



Example:

"Can I apply for this scheme?"

The system should answer from the official scheme document rather than relying only on general model knowledge.

7. Database Architecture

PostgreSQL

- users
- schemes
- scheme_versions
- eligibility_rules
- benefits
- required_documents
- user_profiles
- processing_jobs
- audit_logs

Vector Database / pgvector

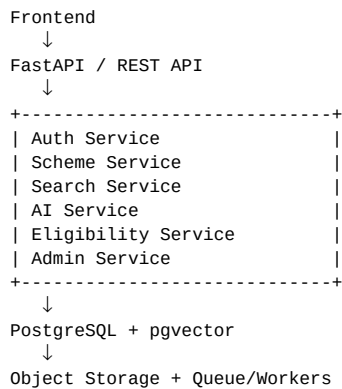
- scheme_id
- document_id
- chunk_id
- embedding
- text

- page_number
- metadata

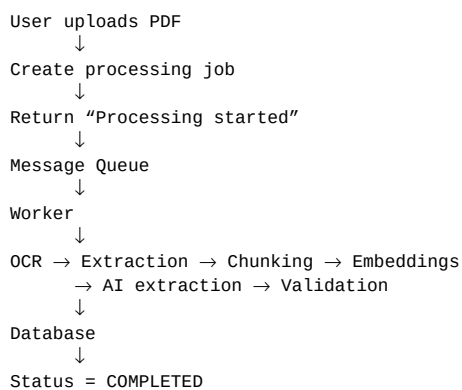
Object Storage

- original PDFs
- extracted text
- metadata
- processed files

8. Backend Architecture



9. Asynchronous Processing



This prevents users from having to wait for long document processing requests.

10. Frontend UX

Home screen:

- Search government schemes
- Upload a scheme PDF
- Browse categories
- Personalized recommendations

Scheme page:

- Scheme name
- Who is it for?
- Benefits

- Eligibility
- Documents required
- How to apply
- Important conditions
- Official source
- Source/page citations

The interface should be simpler than a generic chatbot and focus on actionable information.

11. Key Feature: “Am I Eligible?”

Ask the user:

- Age
- State
- Occupation
- Annual income
- Gender
- Category
- Student / farmer / business status
- Other scheme-specific conditions

Then compare the profile against every mandatory rule.

Important: show explicit rule-by-rule results. A “profile match” score should not be presented as a legal determination of eligibility.

12. Admin Architecture

Admin Dashboard

|

- +-- Add Scheme
- +-- Upload Document
- +-- Review AI Extraction
- +-- Approve Scheme
- +-- Edit Extracted Data
- +-- View Processing Errors
- +-- Manage Versions
- +-- Verify Sources

Recommended workflow:

AI Extraction → Validation → Human/Admin Review → Published

13. Versioning

Government schemes can change.

Do not overwrite old documents.

Scheme

■■■ Version 1 – 2025 Guidelines

■■■ Version 2 – 2026 Guidelines
■■■ Version 3 – Revised Guidelines

Store:

- scheme_id
- version
- effective_from
- effective_until
- source_url
- document_hash
- published_date

14. Security & Reliability

- Verify information against official government sources.
- Preserve the original source URL.
- Attach important claims to document/page citations.
- Store extraction confidence internally.
- Flag uncertain information for review.
- Maintain audit logs for administrative changes.
- Never blindly publish unverified AI extraction.

15. Recommended Technology Stack

Frontend:

- Next.js / React
- Tailwind CSS

Backend:

- Python
- FastAPI

AI:

- LLM API
- Embedding model
- OCR
- Structured-output extraction

Database:

- PostgreSQL
- pgvector

Storage:

- S3-compatible object storage

Async processing:

- Redis + Celery/RQ or cloud queue

Deployment:

- Docker
- AWS / GCP / Azure

16. Development Roadmap

PHASE 1 — MVP

1. Upload government PDF
2. Extract text
3. AI structured extraction
4. Generate concise summary
5. Show benefits, eligibility, documents, application process and conditions
6. Source + page citations
7. Basic search

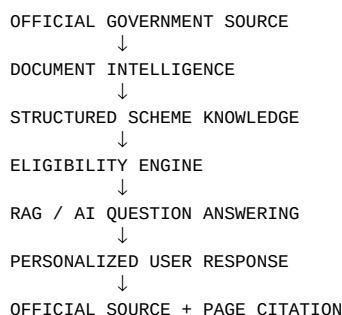
PHASE 2 — Intelligence

- RAG Q&A;
- User profiles
- “Am I eligible?”
- Scheme recommendations
- Scheme comparison
- Multiple languages
- Government website ingestion

PHASE 3 — Production Platform

- Automatic updates
- Version tracking
- Admin verification
- Notifications
- State-specific schemes
- Personalized recommendations
- Analytics
- Mobile app
- Multilingual voice interface

17. Final Architecture



Core positioning:

“Turn complicated government scheme documents into simple, personalized, actionable information.”